Investigación y aplicación de conceptos de Desarrollo de Software

Temas a investigar:

- 11. Saga y transacciones distribuidas - Consistencia
- 12. Base de datos por servicio y propiedad de datos - Persistencia

Grupo 8 - Módulo 9

Integrantes:

- Alcaraz Fabricio Nahuel
- Benítez Agustín Nicolas
- Fernández D'Elia Sol
- López Gustavo Javier
- Lopez Mateo Jeremias
- Machuca Avril Micaela
- Maciel Sanchez Gabriel Angel
- Rosin Gianna Florencia
- Salas Milagros Nadir
- Sanabria Cayre Ernesto Pablo

Materia: Desarrollo Software

Año 2026 - Segundo Cuatrimestre

1. Fundamentos Técnicos

Tema 11: Saga y Transacciones Distribuidas

En arquitecturas distribuidas, el uso de transacciones ACID monolíticas es inviable debido al bloqueo de recursos en bases de datos separadas. El patrón Saga coordina una secuencia de transacciones locales donde cada paso actualiza un servicio y emite eventos o comandos. Si un paso falla, se ejecutan acciones compensatorias inversas para restaurar la consistencia eventual del sistema.

Características Clave

- Consistencia Eventual: Los datos no son consistentes de manera inmediata, pero el sistema converge a un estado consistente de forma asíncrona.
- Coordinación (Dos enfoques principales):
    - Coreografía: Los servicios interactúan intercambiando eventos sin un controlador central.
    - Orquestación: Existe un componente central (Saga Coordinator) que dicta las órdenes a cada servicio mediante comandos y lleva el registro del estado global.
- Idempotencia: Como los mensajes pueden reintentarse ante fallos de red, las operaciones individuales deben ser idempotentes para evitar duplicaciones o inconsistencias si se ejecutan más de una vez.

Beneficios y Limitaciones

- Beneficios: Alta escalabilidad, desacoplamiento entre microservicios, evita bloqueos distribuidos costosos y permite mantener la disponibilidad ante fallos parciales.
- Limitaciones: Complejidad de diseño elevado, dificultad para depurar flujos distribuidos, y la necesidad obligatoria de programar lógica de compensación (reversiones lógicas).

Tema 12: Base de Datos por Servicio y Propiedad de Datos

El patrón Database per Service establece que cada microservicio debe poseer su propia base de datos. Ningún otro servicio tiene permiso para acceder directamente a las tablas o colecciones internas ajenas mediante consultas directas. Resuelve el antipatrón de la base de datos compartida (Shared Database), el cual genera un acoplamiento estrecho donde cualquier modificación en el esquema de una tabla por parte de un equipo rompe inadvertidamente la funcionalidad de los demás servicios.

Características Clave

- Encapsulamiento de Datos: El acceso a los datos de un servicio se realiza exclusivamente a través de su API pública o mediante eventos de dominio.
- Arquitectura Políglota: Permite que cada microservicio elija el motor de base de datos que mejor se adapte a su problema (ej. relacional PostgreSQL para pagos, NoSQL MongoDB para historial de viajes, Redis para caché de geolocalización).
- Autonomía de Equipos: Los desarrolladores de un servicio pueden modificar estructuras internas, índices o esquemas sin coordinarse con otros equipos.

Beneficios y Limitaciones

- Beneficios: Desacoplamiento total, independencia de despliegue, mejor escalabilidad del almacenamiento y libertad tecnológica.
- Limitaciones: Dificultad para ejecutar consultas complejas o reportes analíticos que involucren múltiples dominios, y la complejidad añadida de mantener la integridad referencial a nivel de aplicación.

2. Análisis Crítico, Ventajas, Limitaciones y Criterios de Uso

| Concepto | Cuándo SÍ Utilizar | Cuándo NO Utilizar |
|---|---|---|
| Sagas / Transacciones Distribuidas | Procesos de negocio complejos que cruzan múltiples microservicios independientes (ej. pagos, reservas, logística).<br><br>Cuando se prioriza la disponibilidad y la escalabilidad horizontal por encima de la consistencia estricta instantánea. | Operaciones confinadas a un único servicio donde una transacción ACID local resuelve la atomicidad.<br><br>Cuando las acciones de compensación son físicamente imposibles de revertir (ej: realizar un cobro irrevocable en pasarelas externas sin API de reembolso automatizado). |
| Base de Datos por Servicio | Equipos de desarrollo autónomos, despliegues independientes y necesidades heterogéneas de almacenamiento.<br><br>Cuando diferentes partes del sistema tienen patrones de acceso a datos radicalmente distintos | Sistemas pequeños o monolíticos donde la fragmentación introduce sobrecarga operativa excesiva.<br><br>Cuando se requiere una consistencia relacional rígida y masiva con múltiples JOIN y restricciones de clave foránea cruzadas entre entidades de negocio fuertemente relacionadas. |

3. Alternativas Tecnológicas

- Sagas y Transacciones Distribuidas
    - Two-Phase Commit (2PC): Descartado en arquitecturas nativas de la nube por su alto acoplamiento y bloqueo de recursos, pero útil en sistemas distribuidos estrictamente controlados.
    - Outbox Pattern / Event-Driven Architecture: Para consistencia eventual basada puramente en eventos sin requerir una máquina de estados compleja de compensación.
- Base de Datos por Servicio
    - Database-per-Schema: Compartir una misma instancia o motor de base de datos física pero aislar los datos mediante esquemas separados (útil para reducir costos de infraestructura en etapas iniciales).

4. Vinculación con el TPI

4.1. Aplicación de Sagas (Tema 11) - Simulación del Flujo de Solicitud de Viaje

Podríamos implementar una Saga Orquestada para el flujo crítico de Solicitud y Asignación de Viaje:

- Paso 1 (Viajes): Se registra el viaje con estado inicial PENDIENTE_PAGO.
- Paso 2 (Pagos): Se procesa la retención temporal de fondos en la pasarela. Estado: PAGO_AUTORIZADO.
- Paso 3 (Conductores): Se intenta asignar un conductor disponible cercano. El servicio asigna al chofer y actualiza el estado a CONFIRMADO.

Escenario de fallo: Si en el Paso 3 no se encuentran conductores disponibles en la zona, el orquestador detecta el error y ejecuta de forma inversa las acciones compensatorias:

1. Se ordena al Servicio de Pagos liberar la retención de fondos realizada en el Paso 2.
2. Se actualiza el registro en el Servicio de Viajes a CANCELADO_POR_FALTA_DE_CONDUCTORES.

Esto garantiza la consistencia eventual y ofrece total trazabilidad visual en los registros de auditoría del sistema.

- Visualización: Mediante logs o una consola de monitoreo del orquestador, se debería poder visualizar la transición de estados (PENDIENTE_PAGO ➔ PAGO_AUTORIZADO ➔ ERROR_ASIGNACION ➔ COMPENSANDO_PAGO ➔ VIAJE_CANCELADO), cumpliendo con la exigencia técnica de trazabilidad y recuperación ante fallos distribuidos.

4.2. Aplicación de Base de Datos por Servicio (Tema 12)

Podríamos hacer que la arquitectura de la plataforma se descomponga en microservicios funcionales independientes, tales como:

1. Servicio de Pasajeros: Gestiona perfiles, métodos de pago y preferencias de los usuarios. (BD_Pasajeros)
2. Servicio de Conductores: Gestiona la información del chofer, estado de actividad, vehículos registrados y documentación. (BD_Conductores)
3. Servicio de Viajes: Administra la creación de solicitudes, cotizaciones de tarifas y ciclo de vida del viaje. (BD_Viajes)
4. Servicio de Pagos: Procesa transacciones, retenciones de saldo y comisiones. (BD_Pagos)

Entonces cuando el Servicio de Viajes necesita mostrar los datos del conductor asignado a un pasajero, no realiza un JOIN con la tabla de conductores del Servicio de Conductores. En su lugar, el Servicio de Viajes consulta la información mediante una llamada a la API REST del Servicio de Conductores o bien consume eventos publicados previamente (ej. ConductorAsignadoEvent), almacenando una vista proyectada localmente si es necesario para optimizar lecturas. Esto permite modificar el esquema interno de la base de datos de conductores sin afectar en absoluto al servicio de viajes.

5. Bibliografía

[1] C. Richardson, Microservices Patterns: With examples in Java. Manning Publications, 2018.

- Capítulo 3: Inter-process communication in a microservice architecture (Sagas y comunicación asíncrona).
- Capítulo 4: Managing transactions with sagas (Coordinación de coreografía y orquestación, acciones compensatorias).
- Capítulo 5: Designing business logic in a microservice architecture (Database per Service y propiedad de datos).

[2] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. O'Reilly Media, 2021. (Secciones sobre gestión descentralizada de datos y transacciones distribuidas en microservicios).

[3] M. Fowler, Patterns of Enterprise Application Architecture. Addison-Wesley, 2002. (Fundamentos sobre patrones de mapeo de datos, separación de dominios y arquitecturas orientadas a servicios).

[4] Microsoft Azure Architecture Center. Patrón Saga. Recuperado de https://learn.microsoft.com/es-es/azure/architecture/patterns/saga

[5] Microsoft Learn. Soberanía de datos por microservicio (Data sovereignty per microservice). Recuperado de https://learn.microsoft.com/es-es/dotnet/architecture/microservices/architect-microservice-container-applications/data-sovereignty-per-microservice

[6] AWS Prescriptive Guidance. Database per service pattern. Amazon Web Services. Recuperado de https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-data-persistence/database-per-service.html

[7] Microservices.io. Pattern: Database per service. Chris Richardson. Recuperado de https://microservices.io/patterns/data/database-per-service.html

[8] Conduktor. Saga Pattern for Distributed Transactions. Recuperado de https://www.conduktor.io/glossary/saga-pattern-for-distributed-transactions

[9] Google Cloud. ¿Qué es la arquitectura de microservicios?. Recuperado de https://cloud.google.com/learn/what-is-microservices-architecture?hl=es

Entregables

- [x] ~~Documento o presentación de investigación: fundamentos, análisis de ventajas y desventajas, escenarios de uso y no uso, y vinculación con el Trabajo Práctico Integrador.~~
- [ ] Repositorio de código: implementación funcional de ambos conceptos, con instrucciones de instalación, ejecución y pruebas.
- [ ] Demostración práctica: ejecución de una solución integrada que permita observar y verificar el comportamiento de los conceptos investigados.
- [ ] Coloquio: presentación y defensa de la investigación y la implementación, con preguntas conceptuales y técnicas a los integrantes.

Para llevar a la práctica la implementación y la demostración de tu informe (enfocado en el flujo de solicitud de viaje con el patrón Saga Orquestada y Database per Service), aquí tienes ideas concretas de diseño y código para estructurar tu repositorio:

1. Arquitectura del Repositorio (Estructura de Carpetas)
Para cumplir con la independencia de bases de datos, organiza tu proyecto como un monorepo con servicios desacoplados:

```
None

/tpi-grupo8-modulo9
│
├── /saga-orchestrator       # Coordinador de la Saga (Máquina de estados)
├── /travel-service          # Servicio de Viajes + Base de datos propia
(ej. PostgreSQL / MongoDB)
├── /payment-service         # Servicio de Pagos + Base de datos propia
├── /driver-service          # Servicio de Conductores + Base de datos
propia
├── docker-compose.yml       # Orquestación de contenedores y BDs
independientes
└── README.md                # Instrucciones de instalación y ejecución
```

2. Implementación de la Base de Datos por Servicio (Tema 12)
- Aislamiento físico: En el docker-compose.yml, levanta una instancia de base de datos separada (o esquemas independientes en un motor relacional, pero con contenedores/bases distintas para simular el aislamiento real) para cada microservicio:
  - db_travels (para el microservicio de viajes)
  - db_payments (para el microservicio de pagos)
  - db_drivers (para el microservicio de conductores)
- Regla de oro: El código del Servicio de Viajes solo puede conectarse a db_travels. Si necesita saber el nombre del conductor, lo hace consultando la API HTTP del Servicio de Conductores (GET /drivers/{id}) o escuchando eventos, jamás haciendo un JOIN SQL inter-bases.

3. Implementación de la Saga Orquestada y el Flujo de Fallo (Tema 11)
El corazón de la demostración estará en el Orquestador. Puedes programar un endpoint en el orquestador (ej. POST /api/rides/request) que ejecute los pasos de manera secuencial controlando las compensaciones:
A. Estructura del Flujo en Código (Pseudocódigo / Lógica orientada a Node.js/Python/Java):
JavaScript

```
None

async function requestRideSaga(rideData) {
    let paymentAuthorized = false;
```

```
    let driverAssigned = false;

    try {
        // PASO 1: Crear viaje en estado PENDIENTE_PAGO
        const ride = await TravelService.createRide({ ...rideData, status:
'PENDIENTE_PAGO' });

        try {
            // PASO 2: Autorizar pago (Retención de fondos)
            await PaymentService.authorizePayment({ rideId: ride.id,
amount: ride.fare });
            paymentAuthorized = true;
            await TravelService.updateStatus(ride.id, 'PAGO_AUTORIZADO');

            // PASO 3: Asignar Conductor (Aquí simulamos el fallo para la
demo)
            // Agregamos un flag de prueba: si rideData.forceError es
true, simulamos que no hay choferes.
            if (rideData.forceError) {
                throw new Error("ERROR_NO_DRIVERS_AVAILABLE");
            }

            await DriverService.assignDriver({ rideId: ride.id });
            driverAssigned = true;
            await TravelService.updateStatus(ride.id, 'CONFIRMADO');

            return { success: true, message: "Viaje confirmado
exitosamente" };

        } catch (stepError) {
            // SI FALLA EL PASO 3 (O EL 2), DISPARAMOS LAS ACCIONES
COMPENSATORIAS EN ORDEN INVERSO
            console.log(`[SAGA] Error detectado: ${stepError.message}.
Iniciando compensaciones...`);

            if (paymentAuthorized) {
                console.log("[SAGA] Compensación: Revirtiendo / liberando
pago...");
                await PaymentService.refundPayment({ rideId: ride.id });
```

```
            }

            // Actualizar estado final del viaje
            await TravelService.updateStatus(ride.id,
'CANCELADO_POR_FALTA_DE_CONDUCTORES');

            return { success: false, error: "La reserva falló y el sistema
fue compensado exitosamente." };
        }

    } catch (initialError) {
        throw new Error("Fallo crítico al iniciar el viaje en el servicio
de origen.");
    }
}
```

4. Ideas para la Demostración Práctica (Para mostrar en el repo / video / defensa)
Para que el profesor o evaluador vea el comportamiento en vivo, puedes preparar dos escenarios de prueba ejecutables mediante una colección de Postman o un script de prueba (test-flow.sh):

Escenario A: Flujo Exitoso (Todo OK)

1. Haces un POST /api/rides/request con forceError: false.
2. Qué se observa:
    - El servicio de viajes crea el registro.
    - El servicio de pagos retiene el dinero.
    - El servicio de conductores asigna un chofer libre.
    - Estado final en base de datos: CONFIRMADO.

Escenario B: Flujo con Error Forzado y Compensación (El requerimiento del TPI)

1. Haces un POST /api/rides/request con forceError: true.
2. Qué se observa en la consola / logs del Orquestador:
    - Se registra el viaje (PENDIENTE_PAGO).
    - Se autoriza el pago (PAGO_AUTORIZADO).
    - Se intenta asignar conductor ➔ Falla (ERROR_NO_DRIVERS_AVAILABLE).
    - El orquestador salta al bloque catch y ejecuta:
        - [COMPENSACIÓN] Liberando fondos en el servicio de pagos...
        - [ACTUALIZACIÓN] Pasando viaje a CANCELADO_POR_FALTA_DE_CONDUCTORES...
3. Verificación de Consistencia Eventual: Si consultas la base de datos de pagos, verás que el saldo retenido volvió a cero (rollback lógico aplicado mediante compensación), demostrando que el sistema recuperó la consistencia sin usar transacciones distribuidas bloqueantes.

Cuando termine el segundo grupo planteamos el coloquio